

Practice session 4

The following exercises refer to the topics covered in the previous lessons. Some of them will be carried out together with the teacher. Others will have to be carried out individually, and - at the end of the allotted time - the teacher will illustrate and comment on their correct execution. Exercises that will not be carried out in the classroom must in any case be considered an integral part of the Practice, to be carried out independently at home.

The Tutors present in the classroom during the Practice can only provide support in the execution of some steps. Any doubts about the approach used by the teacher and the validity of any alternative solutions should be clarified exclusively with the teacher.

Exercise 1

In a new Python file, named **Exercise 1.py**, create the function **harmonic_sum** which must calculate the value of the following series:

$$\sum_{n=1}^N \frac{1}{n^{\alpha}}$$

For example, considering $N = 6$ and $\alpha = 2$, the function must calculate the sum of the following terms:

$$1 + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \frac{1}{5^2} + \frac{1}{6^2}$$

In particular, the function must:

- have as mandatory parameters N and α , that are the number of terms of the series and the exponent to be applied
- return the sum of all the terms to the calling instruction

Exercise 2

In a new Python file named **Exercise 2.py**, create the function **roll_the_dice**, which allows you to play with three six-sided dice (not rigged). In this game the player chooses the winning number (for example 6) and the number of rolls of the three dice (for example 100); then the dice are rolled and the player gets one point each time the three dice return three numbers equal to the winning number (for example: all the dice return 6 as a result). In particular, the function **roll_the_dice** must:

- have the winning number as a mandatory parameter
- have the number of rolls as an optional parameter (default value 1,000)
- create the empty list *winning_rolls*

- simulate the specified number of rolls using the functions of the **random** module
- store the serial number of the winning rolls in the *winning_rolls* list (e.g. if the three dice are equal to the winning number in rolls n. 40, 150 and 450, *winning_rolls* must contain [40, 150, 450])
- calculate the score obtained by the user, assigning a point each time the result of the three dice is equal to the winning number
- adding the score as the first element of the list *winning_rolls* and return it (i.e. return the final version of *winning_rolls*)

Exercise 3

The file **Exercise 3.py** contains the list *cities* which contains the names of some cities. You are asked to create a function called **create** to create a dictionary with some randomly generated integers as keys and some elements of a sequence as values. In particular, the function must:

- have a sequence and an integer as mandatory parameters
- create a new dictionary, with a number of key-value pairs equal to the integer passed to the function as a mandatory argument, in which keys are random integer numbers between 1 and 100 (both included) and values are chosen randomly from the sequence passed as a mandatory argument (N.B.: the dictionary mustn't contain duplicate keys or duplicate values)
- return the dictionary

Exercise 4

The file **Exercise 4.py** contains the list *cypher* in which we can find all lowercase and uppercase letters, integers from 0 to 9 and some special characters. You are asked to create a function called **crypt** that uses a simple encryption system in which each character of a text string is replaced with the character placed N positions ahead in *cypher*.

In particular, the function must:

- have a text string (the string to be encrypted) as a mandatory parameter
- have, as an optional parameter, the number of positions needed to choose the characters from *cypher*. For example, if this parameter is 10, the character placed 10 positions ahead in *cypher* is chosen; if it is 20, the characters placed 20 positions ahead in *cypher* is chosen; etc.
- replace each character in the text string to be encrypted with the corresponding characters extracted from *cypher*
- if the character to be encrypted is not in *cypher* list, the same character must be applied to the encrypted string
- return the encrypted text string

Note: We recommend using indexing to find and replace characters. If it is not possible to use a character placed N positions ahead in *cypher*, use the character placed N positions behind.

Exercise 5

In a new Python file named **Exercise 5.py**, create the function **present_value** to calculate the present value of an investment consisting in a series of constant annual payments (CF) for a specified duration (in years) and for a specific constant interest rate (r). To calculate the present value, the following formula must be applied:

$$\sum_{n=1}^N \frac{CF}{(1+r)^n}$$

For example, considering a duration of 4 years ($N = 4$), a constant periodic payment equal to 200 euros ($CF = 200$) and an annual interest rate equal to 5% ($r = 0.05$) the result will be:

$$\frac{200}{(1+0.05)^1} + \frac{200}{(1+0.05)^2} + \frac{200}{(1+0.05)^3} + \frac{200}{(1+0.05)^4} = 709.19$$

The function must have the annual interest rate, the duration of the investment (expressed in years) and the periodic payment as mandatory parameters and it must return the present value.